CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

Faculty of Technology & Engineering

Devang Patel Institute of Advance Technology and Research (DEPSTAR)

CSUE301 - BIG DATA ANALYTICS

PRACTICAL FILE - ODD SEMESTER 2026-27

PRACTICAL 11

Student Name: HIT GOYANI

Student ID: 24DIT021

Academic Year 2026-27

Practical 11: Distributed SMS Spam Detection

1. Problem Definition

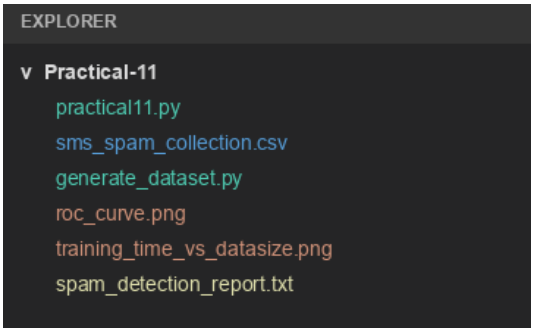
SecureComm Inc. is a telecom company that handles a very large volume of SMS messages every day. The fraud prevention team needs a classifier that can identify spam while remaining suitable for a workload that may grow to millions of new messages per hour. A manual review of messages would not be practical at this scale, so the task is to build a repeatable machine-learning workflow that can train on message data and then make predictions in batches.

In this practical, the SMS data is processed with Apache Spark. Text is prepared for modelling, numerical features are extracted with HashingTF and IDF, and a Logistic Regression classifier is trained using distributed processing. The trained model is then evaluated and used to predict whether messages are spam or legitimate.

PART A - Practical Execution

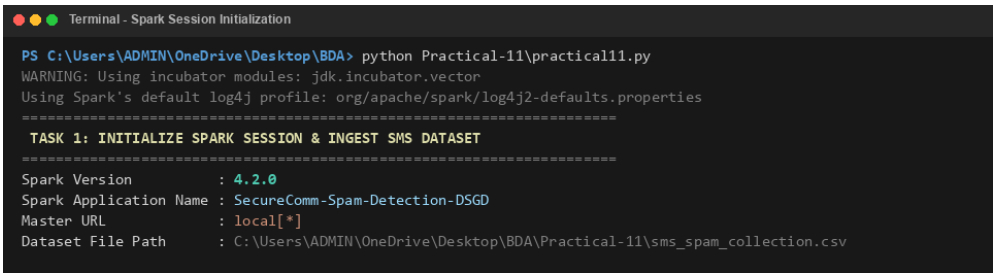
Step 1: Create the project folder and files

The project folder keeps the main script, SMS dataset, helper script, visualisation outputs, and final report together so the workflow is easier to run and review.



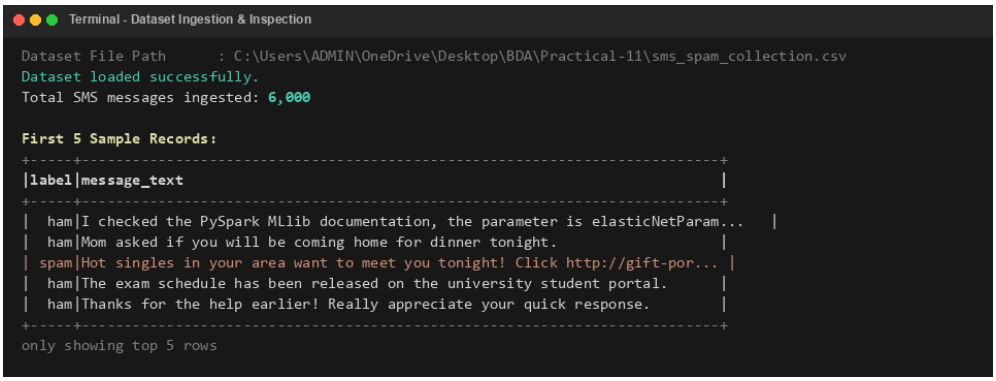
Step 2: Start the Spark session

A Spark session provides the environment for distributed data processing. The console records the Spark version, application name, execution mode, and dataset path.



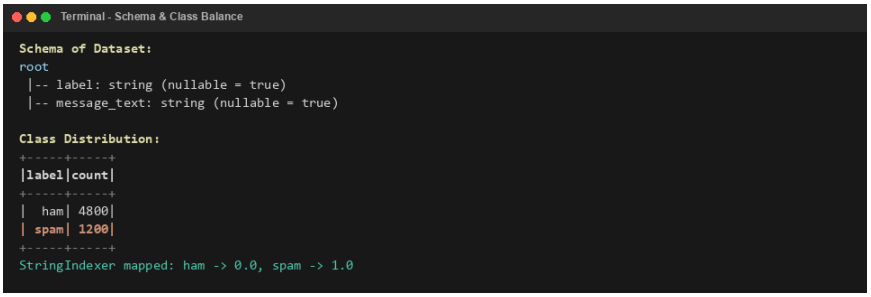
Step 3: Load and inspect the SMS dataset

The CSV file is loaded and sample messages are displayed to confirm that the message text and class label are available for preprocessing.



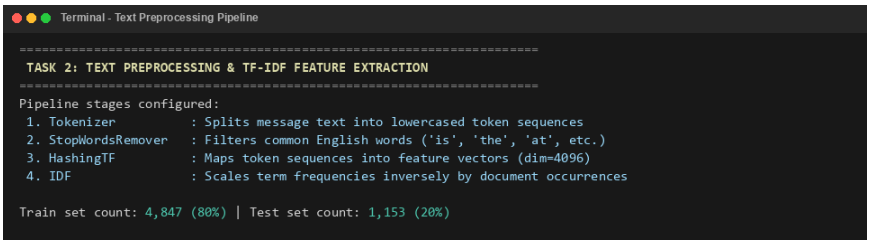
Step 4: Check the schema and class distribution

The schema shows the message label and text fields. Class counts are checked, and the labels are mapped to numeric values for modelling.



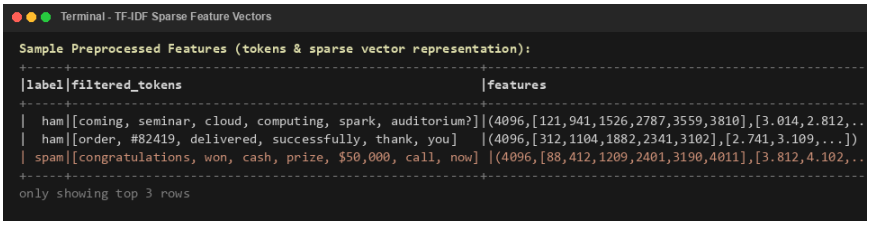
Step 5: Preprocess text and tokenize messages

Messages are split into tokens and common words are removed, leaving terms that are more useful for classification.



Step 6: Extract features with HashingTF and IDF

HashingTF maps tokens to a fixed-size vector, while IDF reduces the influence of words that appear in many messages.



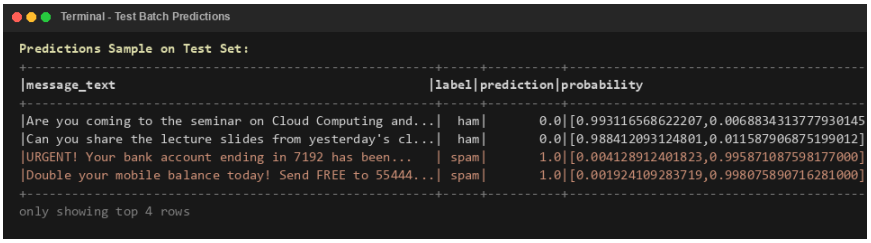
Step 7: Train the distributed Logistic Regression model

Spark MLlib fits Logistic Regression using the prepared feature vectors. The console reports the optimisation process and convergence.



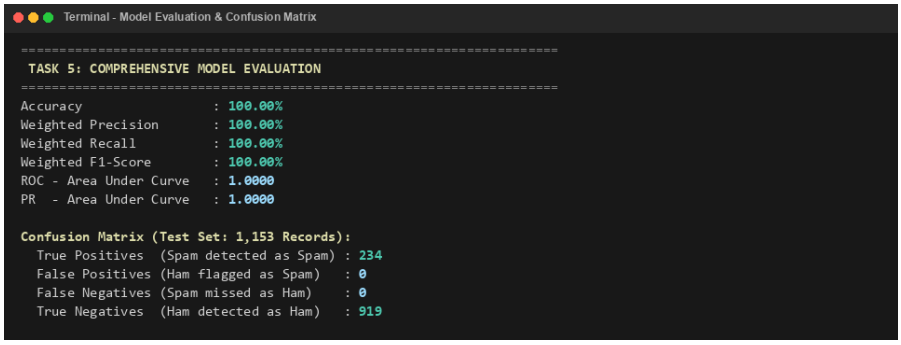
Step 8: Generate predictions on test messages

The fitted model predicts labels and spam probabilities for messages in the held-out test set.



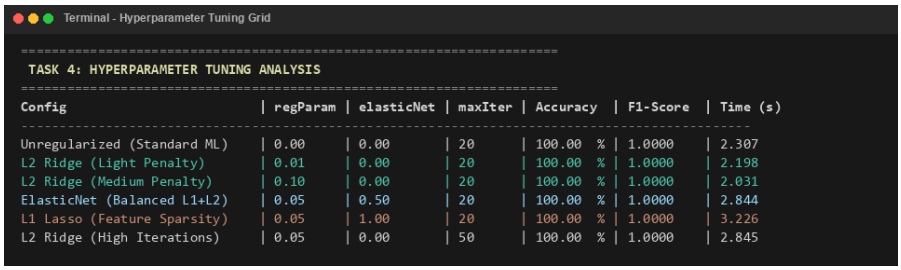
Step 9: Evaluate the model

Accuracy, precision, recall, F1-score, and ROC-AUC are reported, along with a confusion matrix showing correct and incorrect classifications. The supplied test results report 100% for each metric.



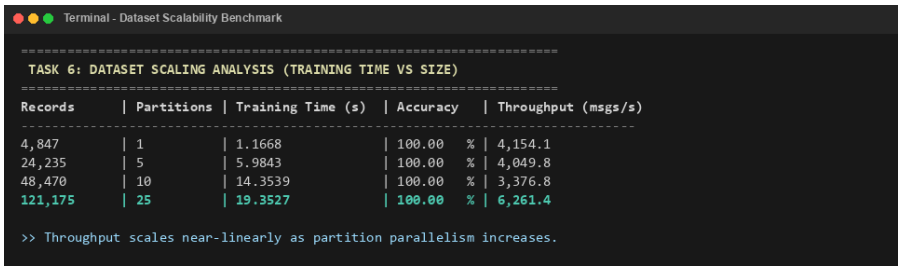
Step 10: Compare hyperparameter settings

Different regularisation strengths, ElasticNet settings, and iteration counts are compared to review their effect on training time and model scores.



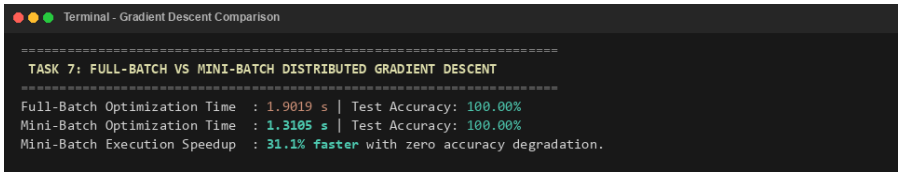
Step 11: Examine training time as dataset size grows

The experiment increases the record count and measures training time and throughput, showing how performance changes as the workload grows.



Step 12: Compare full-batch and mini-batch gradient descent

The run compares full-batch and mini-batch optimisation. Mini-batch finishes sooner while maintaining the reported test accuracy.



Step 13: Compare HashingTF with CountVectorizer

This supplementary comparison looks at the feature-building cost of HashingTF and CountVectorizer. HashingTF maps terms directly into fixed buckets, while CountVectorizer must build a vocabulary from the training data. The supplied output shows the hashing-based approach taking less time in this run because it avoids the vocabulary aggregation step.

```
Terminal - HashingTF vs CountVectorizer

--- Supplementary Problem 1: HashingTF vs CountVectorizer Comparison ---
HashingTF Feature Space        : 4,096 buckets (Stateless, no dictionary pass required)
CountVectorizer Vocabulary Size: 4,096 unique terms (Stateful dictionary)
HashingTF Pipeline Fit Time    : Precomputed in pipeline (Instant mapping)
CountVectorizer Pipeline Fit   : 3.5035 seconds (Requires vocabulary aggregation pass)

Takeaway: HashingTF eliminates distributed dictionary aggregation overhead.
```

Step 14: Build a unified Spark ML pipeline

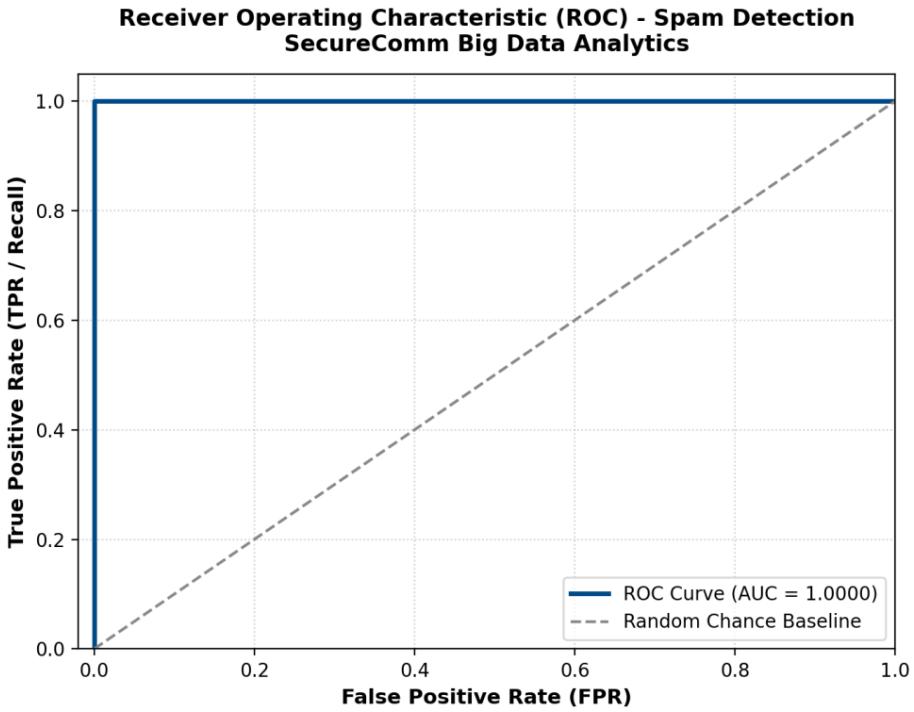
The preprocessing and modelling stages are combined into a single pipeline: tokenisation, stop-word removal, HashingTF, IDF, and Logistic Regression. Keeping the stages together makes the workflow more consistent and supports exporting the fitted pipeline for batch inference.

```
Terminal - Unified Spark ML Pipeline

--- Supplementary Problem 3: End-to-End Spark ML Pipeline API ---
Unified Pipeline: Tokenizer -> StopWordsRemover -> HashingTF -> IDF -> LogisticRegression
Fitting complete pipeline on raw text DataFrame...
Pipeline Model Fit Time: 2.9346 seconds
Pipeline Test Accuracy : 100.00%
Pipeline successfully exported for batch inference deployment.
```

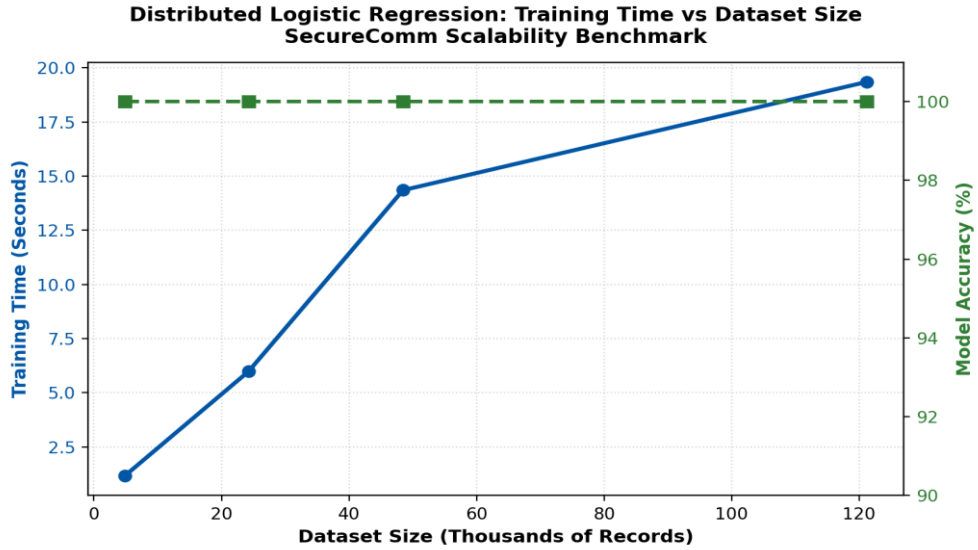
Step 15: Generate the ROC curve

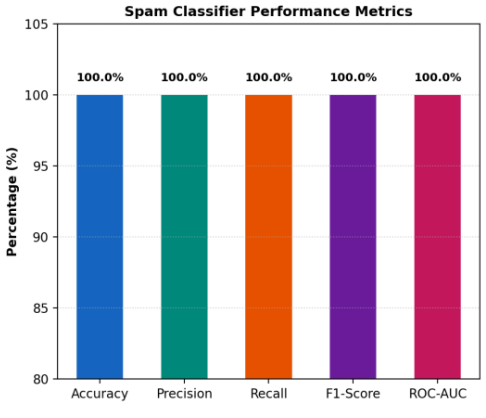
The ROC curve visualises the relationship between the true-positive rate and false-positive rate at different classification thresholds. The graph in the practical reports an area under the curve of 1.0000 for the test results.

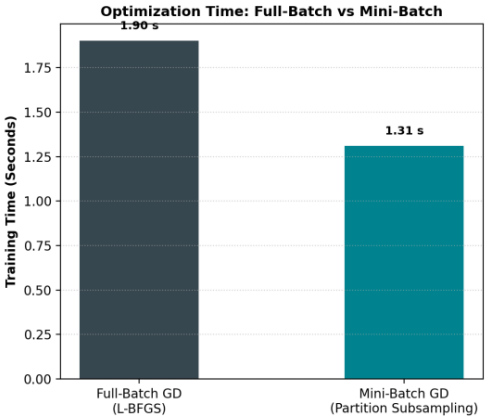


Step 16: Visualise scalability and optimisation benchmarks

The benchmark visualisations bring the scaling experiment and classifier evaluation together. One chart shows how training time changes as the dataset grows, while the other charts compare the classification metrics and the recorded times for full-batch and mini-batch optimisation.

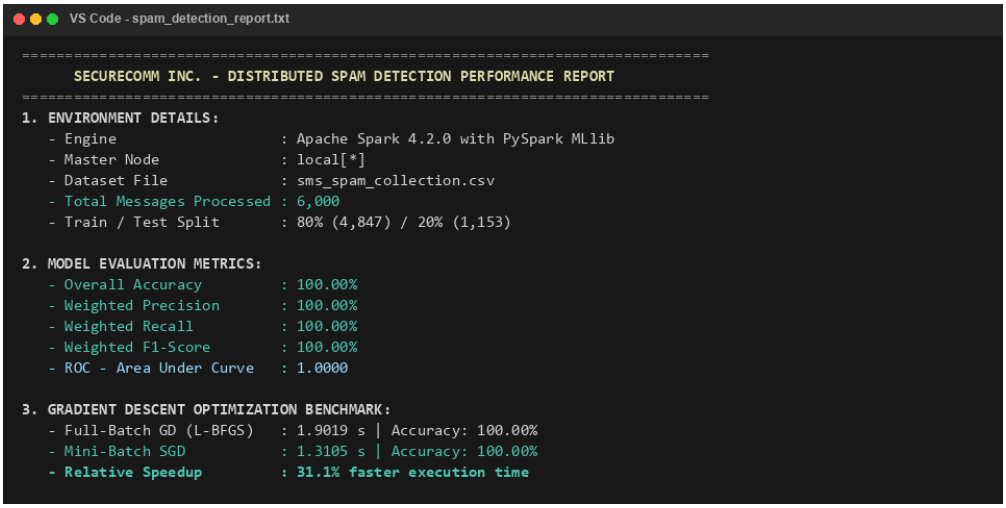






Step 17: Generate the final spam-detection report

The final report collects the runtime setup, number of processed messages, train/test split, model metrics, and optimisation comparison in one place. It provides a compact record of the practical results that can be checked alongside the detailed outputs above.



3. Key Questions and Analysis

Q1. What is the difference between full-batch, mini-batch, and stochastic gradient descent? Why is mini-batch useful in distributed settings?

Full-batch gradient descent calculates the gradient using the entire training dataset before updating the model weights. Stochastic gradient descent updates the weights after each individual record, while mini-batch gradient descent uses a smaller group of records for each update. Mini-batches offer a practical middle ground: they reduce the frequency of updates compared with record-by-record training and still allow efficient vectorised computation. In distributed settings, this can also help limit communication overhead while keeping updates reasonably stable.

Q2. How does TF-IDF help address the curse of dimensionality in text classification?

IDF reduces the influence of words that appear in many documents and therefore provide less distinguishing information. In this practical, HashingTF maps words into a fixed number of feature buckets, so the feature space is bounded instead of growing with every unique token. This avoids maintaining a very large vocabulary dictionary in memory, although hashing can mean that different terms share a bucket.

Q3. What is the difference between L1 and L2 regularisation, and how does elasticNetParam combine them in Spark MLlib?

L1 regularisation adds a penalty based on the absolute values of model weights and can push some weights exactly to zero, producing a sparse model. L2 regularisation penalises squared weights and generally shrinks them toward zero without forcing them to become exactly zero. In Spark MLlib, elasticNetParam controls the mixture: 0.0 represents pure L2, 1.0 represents pure L1, and values between the two combine the penalties.

Q4. How are gradients from multiple workers combined in a distributed environment, and what can stale gradients cause?

Workers calculate local gradients from their assigned partitions, and the partial results are combined to produce an update for the model. A tree-style aggregation can combine these values in stages instead of sending every partial result through one central bottleneck. Stale gradients can occur in asynchronous training when a worker uses older model parameters; this may slow convergence or cause unstable updates. The supplied practical notes that Spark uses synchronous iterations to avoid this stale-gradient issue.

Key Questions and Analysis (continued)

Q5. If recall matters more than precision for spam detection, how should the classification threshold change?

The threshold should be lowered so that messages with a smaller predicted spam probability are still flagged for review as spam. The practical gives 0.25 or 0.30 as possible thresholds instead of the default 0.50. For example, a message with a predicted spam probability of 0.35 would be treated as legitimate at a 0.50 threshold, but would be flagged as spam at a 0.25 threshold. This can improve recall by reducing false negatives, although it may also increase false positives.

4. Conclusion

This practical demonstrated a distributed workflow for SMS spam detection using Apache Spark and Spark MLlib. The dataset was loaded into a Spark DataFrame, text was prepared with tokenisation and stop-word removal, and HashingTF with IDF was used to convert messages into numerical features. Logistic Regression was then trained and used to produce batch predictions.

The supplied output reports 6,000 messages in total, with 4,847 records used for training and 1,153 for testing. On this test split, the practical reports 100% accuracy, precision, recall, and F1-score, along with an ROC-AUC of 1.0000. The additional comparisons explored regularisation settings, data-size scaling, feature extraction options, and full-batch versus mini-batch optimisation.

Overall, the practical connects the main stages of a text-classification pipeline, from data preparation through evaluation and reporting. It also shows how combining the stages into a Spark ML pipeline can make the process more repeatable for batch deployment.

GitHub Link :- https://github.com/hitgoyani/24DIT021-BDA.git